

# One-to-Many Delivery Shipment

---

## 1. What is this feature actually solving?

### Current shipment model

Today, a shipment is:

```
None
ONE Pickup
  ↓
ONE Delivery
```

For example:

```
None
Warehouse
  |
  | 300 units
  ↓
Customer A
```

The shipment is:

```
JSON
{
  "pickup": {
    "id": "P1"
  },
  "delivery": {
    "id": "D1"
  },
  "amount": [300]
}
```

The problem occurs when one pickup needs to distribute its load to **multiple customers**.

For example:

None

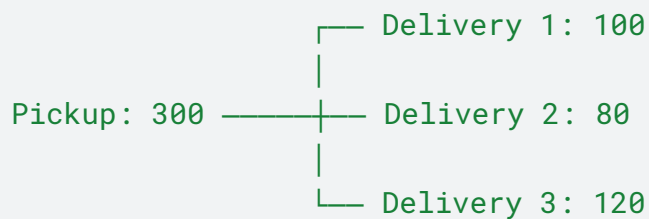


Previously, customers had to model this as multiple shipments sharing the same pickup. The new feature allows this to be represented as **one logical shipment with one pickup and multiple deliveries**.

## 2. The new model

The new structure is:

None



API:

JSON

```
{
  "pickup": {...},

  "deliveries": [
    {...},
    {...},
    {...}
  ],

  "amount": [300]
}
```

The critical rule is:

None

```
SUM(all delivery amounts) = shipment amount
```

So:

None

```
100 + 80 + 120 = 300
```

The document explicitly says this equality must hold dimension-by-dimension.

---

## 3. Very important: This is NOT Split Order

This distinction is important for your testing.

### Split Order

None

```
ONE JOB
```

↓

```
Job split across multiple vehicles
```

### One-to-Many Shipment

None

```
ONE SHIPMENT
```

↓

```
ONE pickup
```

↓

```
MULTIPLE deliveries
```

The pickup is performed **once**, and the vehicle then distributes the picked-up load to multiple delivery locations.

---

## 4. How I understand the optimizer behaviour

Let's use the document's example.

### Input

Pickup:

None

$P1 = 300$

Deliveries:

None

$D1a = 100$

$D1b = 80$

$D1c = 120$

Total:

None

$100 + 80 + 120 = 300$

Vehicle starts with:

None

$load = 0$

At pickup:

None

$load = 300$

After D1a:

None

$300 - 100 = 200$

After D1b:

None

$$200 - 80 = 120$$

After D1c:

None

$$120 - 120 = 0$$

So the route could be:

None

START

↓

P1 Pickup +300

↓

D1a -100

↓

D1b -80

↓

D1c -120

↓

END

The sample response in the document demonstrates exactly this load progression:  $0 \rightarrow 300 \rightarrow 200 \rightarrow 120 \rightarrow 0$ .

---

## 5. Does delivery order matter?

Yes.

The optimizer is allowed to decide the order of the deliveries.

For example, input:

None

$$D1a = 100$$

```
D1b = 80
D1c = 120
```

does **not necessarily** mean:

```
None
P1 → D1a → D1b → D1c
```

It could potentially be:

```
None
P1 → D1c → D1a → D1b
```

provided all relevant constraints are satisfied.

The document says the solver decides the relative order subject to:

- `sequence_order`
- time windows
- `follow_lifo_order`

**So do not write a test expecting the delivery order to match the input array unless Engineering confirms that behaviour.**

---

## 6. The most important business rule

This is probably the #1 assertion you should have.

Given:

```
JSON
"amount": [300]
```

and:

```
JSON
"deliveries": [
```

```
{ "amount": [100] },  
{ "amount": [80] },  
{ "amount": [120] }  
]
```

assert:

```
None  
100 + 80 + 120 = 300
```

For multiple dimensions:

```
JSON  
"amount": [300, 15]
```

and:

```
None  
D1 = [100, 5]  
D2 = [80, 4]  
D3 = [120, 6]
```

Then:

```
None  
Dimension 1:  
100 + 80 + 120 = 300  
  
Dimension 2:  
5 + 4 + 6 = 15
```

The document specifically calls out this multi-dimensional validation.

---

## 7. Most important API rule: exactly ONE of delivery or deliveries

A shipment must have:

None

`delivery`

OR

None

`deliveries`

but **never both**.

And never neither.

### Valid 1-to-1

JSON

```
{
  "pickup": {...},
  "delivery": {...},
  "amount": [100]
}
```

### Valid 1-to-many

JSON

```
{
  "pickup": {...},
  "deliveries": [
    {...},
    {...}
  ],
  "amount": [100]
}
```

### Invalid

JSON

```
{
  "pickup": {...},
  "delivery": {...},
```

```
"deliveries": [  
  {...},  
  {...}  
],  
"amount": [100]  
}
```

Expected:

```
None  
400 Bad Request
```

The same applies if neither is supplied.

---

## 8. Minimum deliveries = 2

This is another important validation.

This is invalid:

```
JSON  
"deliveries": [  
  {  
    "id": "D1",  
    "amount": [100]  
  }  
]
```

Because `deliveries` is specifically for one-to-many and must contain **at least two entries**.

For one delivery, use the existing:

```
JSON  
"delivery": {...}
```

The document explicitly defines this limitation.

---

## 9. Delivery IDs

Every delivery must have an ID.

For example:

JSON

```
"deliveries": [  
  {  
    "id": "D1a"  
  },  
  {  
    "id": "D1b"  
  },  
  {  
    "id": "D1c"  
  }  
]
```

And these IDs must be unique across **all steps in the request**.

So this should fail:

None

```
Pickup ID = P1  
Delivery 1 ID = D1  
Delivery 2 ID = D1
```

Also:

None

```
Pickup ID = P1  
Delivery ID = P1
```

should fail.

Expected:

None

400 Bad Request

## 10. Location\_index

The old shipment has:

JSON

```
"delivery": {  
  "id": "D1",  
  "location_index": 1  
}
```

For one-to-many, `location_index` belongs to every delivery:

JSON

```
"deliveries": [  
  {  
    "id": "D1a",  
    "location_index": 1,  
    "amount": [100]  
  },  
  {  
    "id": "D1b",  
    "location_index": 2,  
    "amount": [80]  
  }  
]
```

You should test:

- valid indexes
- duplicate indexes
- invalid indexes
- pickup and delivery at same location
- multiple deliveries at same location

The design document does not explicitly define expected behaviour for all of these, so some should become Engineering questions.

---

## 11. Full basic test request

I'd start testing with something very simple.

JSON

```
{
  "shipments": [
    {
      "pickup": {
        "id": "P1",
        "location_index": 0,
        "service": 600
      },
      "deliveries": [
        {
          "id": "D1a",
          "location_index": 1,
          "amount": [100],
          "service": 300
        },
        {
          "id": "D1b",
          "location_index": 2,
          "amount": [80],
          "service": 300
        },
        {
          "id": "D1c",
          "location_index": 3,
          "amount": [120],
          "service": 300
        }
      ],
      "amount": [300]
    }
  ]
}
```

This is essentially the example provided in the design document.

---

## 12. What should you verify in the response?

For the above request:

### A. Pickup exists

None

P1

### B. All deliveries exist

None

D1a

D1b

D1c

### C. Pickup occurs first

None

P1

↓

D1x

### D. Every delivery happens after pickup

Never:

None

D1a

↓

P1

### E. Load accounting

Expected:

None

Start = 0

Pickup = 300

After D1a = 200

After D1b = 120

After D1c = 0

## F. Final load

None

0

## G. All activities are on the same vehicle

This is extremely important.

A shipment is a joint assignment.

You should NOT get:

None

Vehicle 1

P1

D1a

D1b

Vehicle 2

D1c

The design says all activities in the one-to-many shipment must be assigned or unassigned together.

---

# 13. Capacity testing

This is one of your most important scenarios.

Suppose:

None

Shipment amount = 300

Vehicle capacity = 200

Then pickup requires:

None

300

but vehicle can carry only:

None

200

Expected:

None

Shipment = UNASSIGNED

And importantly:

None

P1 → unassigned

D1a → unassigned

D1b → unassigned

D1c → unassigned

Not:

None

P1 → assigned

D1a → assigned

D1b → assigned

D1c → unassigned

The design explicitly requires **joint assignment**.

---

## 14. Very important distinction: no partial drops

Suppose:

None

D1a = 100

The solver cannot do:

None

D1a → deliver 60

and leave:

None

40

behind.

Each delivery is atomic.

It must deliver:

None

100

or the delivery cannot be partially fulfilled.

The design explicitly states there are **no partial drops within a delivery step**.

---

## 15. max\_time\_in\_vehicle

This deserves dedicated testing.

Suppose:

```
None
Pickup
  ↓ 10 min
D1
  ↓ 10 min
D2
  ↓ 60 min
D3
```

and:

```
None
max_time_in_vehicle = 30 minutes
```

If D3 would take longer than allowed from pickup departure:

```
None
Pickup → D3 > 30 mins
```

then the **entire shipment should become unassigned**, not just D3.

The document explicitly says this is evaluated per delivery and that joint assignment applies.

---

## 16. follow\_lifo\_order

This is another major area to test.

Imagine:

None

Shipment A

P-A

└─ DA1

└─ DA2

Shipment B

P-B

└─ DB1

└─ DB2

With:

JSON

```
"follow_lifo_order": true
```

the document says deliveries belonging to the later shipment should respect the generalized LIFO behaviour.

This needs a dedicated multi-shipment route test.

---

## 17. Compartment mode

This is explicitly unsupported.

If request contains:

None

**compartments**

AND:

None

**deliveries**

expect:

None

400 Bad Request

The design explicitly calls this a Stage 1 limitation.

Also test the reverse:

None

compartments + normal 1-to-1 shipment

because the limitation is specifically around shipments with the new delivery structure.

## 19. Backward compatibility testing

This should be a mandatory regression test.

Existing request:

JSON

```
{
  "pickup": {...},
  "delivery": {...},
  "amount": [100]
}
```

must continue working exactly as before.

The design explicitly states this is an **additive, backward-compatible change**, and the existing 1-to-1 shipment test suite should run unchanged.

I would therefore run your entire existing shipment regression suite.

## 20. Re-optimization testing

This is specifically mentioned in the document.

Test:

### Step 1

Run an existing optimization.

## Step 2

Use:

```
None  
shipment_changes.add
```

to add a new one-to-many shipment.

## Step 3

Re-optimize.

Verify:

```
None  
Pickup  
+  
all deliveries
```

are correctly incorporated into the route.

Also test:

```
None  
shipment_changes.modify
```

and:

```
None  
shipment_changes.remove
```

The document says these accept the new `deliveries` structure and that `modify` **fully replaces** the shipment body rather than partially merging it.

---

# 21. My recommended complete test suite

I'd structure your QA testing like this.

## A. Basic Functional

| TC | Test                                    |
|----|-----------------------------------------|
| 01 | Basic 1 pickup + 2 deliveries           |
| 02 | 1 pickup + 3 deliveries                 |
| 03 | 1 pickup + many deliveries              |
| 04 | Different delivery locations            |
| 05 | Multiple deliveries at same location    |
| 06 | Delivery order differs from input order |
| 07 | Verify all deliveries after pickup      |
| 08 | Verify final load = 0                   |

---

## B. Amount Validation

| TC | Test                             |
|----|----------------------------------|
| 09 | Correct sum                      |
| 10 | Sum greater than shipment amount |
| 11 | Sum less than shipment amount    |
| 12 | Zero delivery amount             |
| 13 | Negative delivery amount         |
| 14 | Missing delivery amount          |
| 15 | Different dimensionality         |
| 16 | Multi-dimensional correct sum    |
| 17 | Multi-dimensional incorrect sum  |

Expected invalid cases should produce 400 where specified.

---

## 22. Input Validation Tests

| TC | Scenario                       | Expected |
|----|--------------------------------|----------|
| 18 | delivery only                  | Success  |
| 19 | deliveries only                | Success  |
| 20 | both provided                  | 400      |
| 21 | neither provided               | 400      |
| 22 | deliveries=[]                  | 400      |
| 23 | deliveries with 1 item         | 400      |
| 24 | delivery missing ID            | 400      |
| 25 | duplicate delivery IDs         | 400      |
| 26 | pickup ID = delivery ID        | 400      |
| 27 | duplicate IDs across shipments | 400      |
| 28 | delivery amount missing        | 400      |
| 29 | wrong amount dimension         | 400      |

The majority of these validation expectations are explicitly defined in the design.

---

## 23. Routing / Optimization Tests

| TC | Scenario                                 |
|----|------------------------------------------|
| 30 | Enough vehicle capacity                  |
| 31 | Insufficient vehicle capacity            |
| 32 | Capacity exactly equals shipment amount  |
| 33 | Capacity exactly equals largest delivery |
| 34 | Multiple shipments on same vehicle       |
| 35 | Multiple one-to-many shipments           |
| 36 | Shipment with time windows               |

- 37 Tight max\_time\_in\_vehicle
  - 38 Impossible max\_time\_in\_vehicle
  - 39 follow\_lifo\_order=false
  - 40 follow\_lifo\_order=true
  - 41 Different delivery order based on optimization
  - 42 Impossible route
- 

## 24. Constraint inheritance tests

The document says shipment-level fields continue to apply at shipment level and **cannot be overridden per delivery**. This is important.

Test:

```
None
skills
priority
zones
load_types
revenue
outsourcing_cost
follow_lifo_order
volume
joint_order
```

For each:

```
None
Shipment-level constraint
    ↓
Pickup + Delivery 1
    ↓
Delivery 2
    ↓
Delivery 3
```

Verify that the solver respects the shipment-level constraint across the entire shipment.

---

## 25. Task count testing

This is easy to miss.

A one-to-many shipment with:

None

$K$  deliveries

counts as:

None

$1 + K$

tasks.

So:

None

$1 \text{ pickup} + 3 \text{ deliveries} = 4 \text{ tasks}$

The global limit is:

None

$4000 \text{ tasks}$

Test around the boundary.

### Example

None

$3999 \text{ tasks}$

should work if otherwise valid.

None

4000 tasks

should work if the limit is inclusive.

None

4001 tasks

should be rejected.

**But I would confirm with Engineering whether 4000 is inclusive.**

---

## 26. Sample ReAPI/Postman assertions

Since you use API automation, I'd create assertions around the response rather than only checking HTTP 200.

For a basic one-to-many shipment, conceptually:

JavaScript

```
// Find the route
const route = response.data.result.routes[0];

// Find pickup
const pickup = route.steps.find(s => s.id === "P1");

// Find deliveries
const deliveries = route.steps.filter(
  s => ["D1a", "D1b", "D1c"].includes(s.id)
);

// Pickup must exist
expect(pickup).to.not.equal(undefined);

// All deliveries must exist
expect(deliveries.length).to.equal(3);

// Pickup must occur before every delivery
```

```
const pickupIndex = route.steps.findIndex(
  s => s.id === "P1"
);

deliveries.forEach(delivery => {
  const deliveryIndex = route.steps.findIndex(
    s => s.id === delivery.id
  );

  expect(deliveryIndex).toBeGreaterThan(pickupIndex);
});
```

---

## 27. Dynamic load validation

I'd also create a generic assertion rather than hardcoding expected route order.

For example:

JavaScript

```
const pickupAmount = 300;

const deliveryAmounts = deliveries.map(
  d => d.delivery?.[0] ?? d.amount?.[0]
);

const totalDelivered = deliveryAmounts.reduce(
  (sum, amount) => sum + amount,
  0
);

expect(totalDelivered).toEqual(pickupAmount);
```

And if you're validating route loads:

JavaScript

```
const finalStep = route.steps[route.steps.length - 1];
```

```
expect(finalStep.load[0]).to.equal(0);
```

This is much better than asserting:

None

```
D1a must be first  
D1b must be second  
D1c must be third
```

because the solver is allowed to determine delivery order.

---

## 28. Negative API test examples

Both **delivery** and **deliveries**

JSON

```
{  
  "shipments": [  
    {  
      "pickup": {  
        "id": "P1",  
        "location_index": 0  
      },  
      "delivery": {  
        "id": "D1",  
        "location_index": 1  
      },  
      "deliveries": [  
        {  
          "id": "D1a",  
          "location_index": 1,  
          "amount": [50]  
        },  
        {  
          "id": "D1b",  
          "location_index": 2,  
          "amount": [50]  
        }  
      ]  
    }  
  ]  
}
```

```
        "amount": [50]
      }
    ],
    "amount": [100]
  }
]
```

Expected:

None

HTTP 400

and message equivalent to:

None

Shipment must specify exactly one of delivery or deliveries

---

## Sum mismatch

JSON

```
"amount": [300],

"deliveries": [
  {
    "id": "D1",
    "location_index": 1,
    "amount": [100]
  },
  {
    "id": "D2",
    "location_index": 2,
    "amount": [100]
  }
]
```

Here:

None

`100 + 100 != 300`

Expected:

None

`400`

The design explicitly defines this validation.

---

## 29. One particularly important test

I'd make this a mandatory test:

**Partial assignment must NEVER happen**

Input:

None

`Pickup = 300`

`D1 = 100`

`D2 = 100`

`D3 = 100`

Create a route where:

None

`D1 = feasible`

`D2 = feasible`

`D3 = impossible`

Expected:

None

`Pickup → UNASSIGNED`

`D1 → UNASSIGNED`

D2 → UNASSIGNED  
D3 → UNASSIGNED

NOT:

None  
Pickup → Assigned  
D1 → Assigned  
D2 → Assigned  
D3 → Unassigned

This is one of the most important business rules in this feature.

---

## 30. Questions

This is the part I'd recommend you take directly into your refinement/QA discussion.

### API / Validation

1. What happens when two deliveries have the same `location_index`?
  - a. Normally it is a bad practice to do so as they can be combined into a regular delivery but we don't return a warning message and they will be treated as two separate delivery steps, normally to minimize total travel cost these two deliveries would be next to each other in response route if there are no further restrictions.
2. [Re-Opt] For `shipment_changes.modify`, does replacing the shipment completely remove the previous deliveries?
  - a. ReOpt does not support one to many deliveries in current version, so no remove, change, or add is supported.
3. Does `shipment_changes.remove` always remove the whole logical shipment, or can an individual delivery be removed independently?
  - a. ReOpt does not support one to many deliveries in current version, so no remove, change, or add is supported.
4. If the original vehicle is removed after completing the pickup but before completing all deliveries, should the request return `400`, become infeasible, or support cargo transfer to another vehicle?
  - a. This question is odd, removing a vehicle means the whole route is not supported, no after completing the pickup or not. There is no difference for the behaviour of removing a vehicle for regular shipment or this one to many delivery shipment.

5. What happens if a delivery is removed during re-optimization?
  - a. ReOpt does not support one to many deliveries in current version, so no remove, change, or add is supported.
6. Does `max_visit_lateness` apply independently to the pickup and to every object in `deliveries[]`, based on each activity's own time window?
  - a. Yes.
7. In case of multi re-run, must all deliveries be completed within the same run as the pickup?
  - a. Yes.

Query about Relation:

On your relation-steps question: neither — relations (including pinned assignments) do not support one-to-many shipments yet, in any spelling. Referencing a delivery group's pickup or any of its deliveries from `relations.steps` returns 400 (e.g. `invalid pinned relation: pickup A2 does not belong to any shipment`). The delivery order within a group is decided by the solver, subject to `sequence_order`, time windows and `follow_lifo_order`. Pinned assignments for one-to-one shipments are unaffected (`"type": "pickup"` / `"type": "delivery"` as today; `"deliveries"` is not a step type — if relation support for delivery groups lands later, each delivery would be referenced individually as `{"type": "delivery", "id": ...}`).